ECE414 – Project Proposal

James Ogle

November 4, 2026

Project Overview

- **Idea:** A rover is placed at a starting position in an unknown grid-based area and is given a goal position to reach. The rover must explore partially observed terrain, choose movement directions, and reach the end goal while accounting for obstacles and stochastic movement failures.
- **Current Implementation:** Currently, the environment is setup to where the rover has a "see-able" area which is highlighted in light grey that adjusts as the robot moves. A basic reward system has also been setup to provide feedback when the rover has a successful move or not.
- **What Remains:** The next stage will be setting up a policy model to work with the environment (ex. Q-learning).

Environment Design

- **Environment:** The environment is an 11×11 grid-based lunar surface containing boundary walls, randomly generated obstacle terrain, a rover start position at $(1, 1)$, and a goal beacon at $(9, 9)$. Breadth-first search verifies that a valid path from the start to the goal exists before each episode begins. The rover initially observes only a local region around its position, while the rest of the terrain remains unknown.
- **State/Observation:** The rover receives a partially observable MiniGrid observation after each action. This observation contains a local encoded view of nearby cells, the rover's facing direction, and the mission text; it does not provide the full map or all obstacle locations. Internally, the environment tracks the rover position, goal position, discovered cells, and the hidden grid layout.
- **Action:** The agent selects one of four discrete actions: move east, south, west, or north. These actions are encoded as 0, 1, 2, and 3, respectively.
- **Reward:** The agent receives a small step penalty of -0.01 to encourage efficient movement. It receives $+0.02$ for each newly revealed cell, $+1.0$ for reaching the goal, -0.05 for attempting to enter an obstacle or wall, and an additional -0.25 if a stochastic movement failure occurs.
- **Transition:** After the rover selects a direction, the environment checks the adjacent target cell. If the cell is a wall or obstacle, the rover remains in its current position; otherwise, it moves to that cell unless a random movement failure occurs, which has a probability of 0.15. After the transition, nearby cells are marked as discovered, a new partial observation is returned, and the episode terminates when the rover reaches the goal or is truncated after the maximum number of steps.

```
❯ nvim lunar_rover_env.py
46    def step(self, action):
45        if not self.action_space.contains(action):
44            raise ValueError(f"Invalid action {action}; expected an integer from 0 through 3.")
43
42        self.step_count += 1
41        dx, dy = self.ACTION_TO_DIRECTION[int(action)]
40        self.agent_dir = int(action)  # keeps rendered rover facing its attempted direction
39        x, y = self.agent_pos
38        candidate = (x + dx, y + dy)
37        cell = self.grid.get(*candidate)
36
35        reward = -0.01  # time/distance penalty
34        movement_failed = False
33        hit_obstacle = cell is not None and not cell.can_overlap()
32
31        if hit_obstacle:
30            reward -= 0.05
29        elif self.np_random.random() < self.movement_failure_probability:
28            # Models getting stuck or an unforeseen terrain problem. The rover stays put.
27            movement_failed = True
26            reward -= 0.25
25        else:
24            self.agent_pos = candidate
```

(a) First Part of step function in environment

```
22        newly_discovered = self._sensor_cells() - self.discovered_cells
21        self.discovered_cells.update(newly_discovered)
20        reward += 0.02 * len(newly_discovered)  # information-gain reward
19
18        reached_goal = self.agent_pos == self.goal_pos
17        if reached_goal:
16            reward += 1.0
15
14        terminated = reached_goal
13        truncated = self.step_count >= self.max_steps
12        observation = self.gen_obs()
11        info = {
10            "movement_failed": movement_failed,
9             "hit_obstacle": hit_obstacle,
8             "newly_discovered": len(newly_discovered),
7             "discovered_cells": len(self.discovered_cells),
6             "position": self.agent_pos,
5         }
4
3         if self.render_mode == "human":
2             self.render()
1         return observation, reward, terminated, truncated, info
175
```

(b) Second Part of step function in environment

The code above is the step method of environment class split up into two pictures. The method updates the step-count, applies reward values, and setups new observation information for the next step. The method will also determine if the episode needs to halt or not whether that is from reaching the end goal or the allotted amount of steps setup by the user.

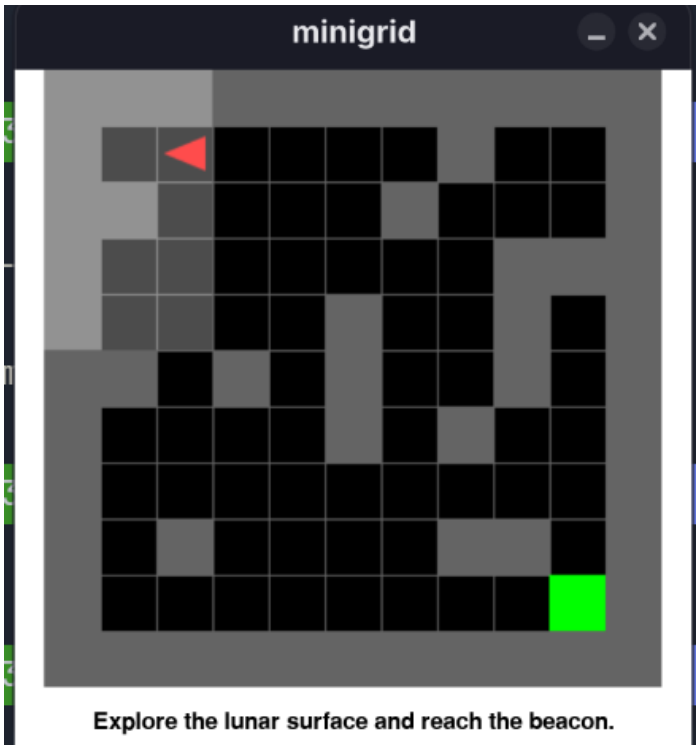


Figure 2: Rendered Lunar Rover environment after initialization. The red rover begins near the upper-left area, the green cell is the goal beacon, black cells are obstacle terrain, and the gray highlighted region represents the rover's local observable area.

Environment Testing

- **Testing Environment:** I tested the environment by running `lunar_rover_env.py`, which creates a `LunarRoverEnv` instance, calls `reset(seed=7)`, and executes 25 randomly sampled actions. During the test, the environment prints the selected action, reward, rover position, number of discovered

cells, collision status, and whether stochastic movement failure occurred. I also visually inspected the rendered MiniGrid window to confirm that walls, obstacles, the rover, and the goal were displayed correctly.

- **Environment Response to Agent Action:** For each action, the `step()` method converts the selected value into one of four cardinal directions. A valid move changes the rover position; an attempted move into a wall or obstacle leaves it in place and applies a collision penalty. Even valid moves can fail with probability 0.15, representing the rover becoming stuck or encountering unknown terrain. The environment then returns the next partial observation, reward, termination flags, and diagnostic information.
- **Correctly Initializing Environment:** The `reset()` method regenerates the grid, places the rover at the fixed start position $(1, 1)$, places the goal at $(9, 9)$ for the default 11×11 grid, and initializes the set of discovered cells around the rover. Obstacle positions are randomly generated, but breadth-first search verifies that at least one path from the start to the goal exists before the episode begins. Using the same seed reproduces the same randomly generated environment for debugging.
- **Technical Challenges/Constraints:** A main challenge was generating random obstacle layouts without producing impossible episodes. This was addressed by using breadth-first search during grid generation to reject layouts without a valid start-to-goal path. Another constraint is partial observability: the rover receives a local MiniGrid observation instead of the entire grid. Finally, stochastic movement introduces uncertainty because a valid action may fail even when the selected cell is reachable.

```
python lunar_rover_env.py
Observation keys: dict_keys(['image', 'direction', 'mission'])
Initial info: {'discovered_cells': 17}
action=2, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 17, 'position': (1, 1)}
action=2, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 17, 'position': (1, 1)}
action=1, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 17, 'position': (1, 1)}
action=3, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 17, 'position': (1, 1)}
action=2, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 17, 'position': (1, 1)}
action=1, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 17, 'position': (1, 1)}
action=2, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 17, 'position': (1, 1)}
action=1, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 17, 'position': (1, 1)}
action=3, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 17, 'position': (1, 1)}
action=3, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 17, 'position': (1, 1)}
action=0, reward=0.09, info={'movement_failed': False, 'hit_obstacle': False, 'newly_discovered': 5, 'discovered_cells': 22, 'position': (2, 1)}
action=1, reward=0.07, info={'movement_failed': False, 'hit_obstacle': False, 'newly_discovered': 4, 'discovered_cells': 26, 'position': (2, 2)}
action=1, reward=0.11, info={'movement_failed': False, 'hit_obstacle': False, 'newly_discovered': 6, 'discovered_cells': 32, 'position': (2, 3)}
action=1, reward=0.11, info={'movement_failed': False, 'hit_obstacle': False, 'newly_discovered': 6, 'discovered_cells': 38, 'position': (2, 4)}
action=0, reward=0.07, info={'movement_failed': False, 'hit_obstacle': False, 'newly_discovered': 4, 'discovered_cells': 42, 'position': (3, 4)}
action=3, reward=0.01, info={'movement_failed': False, 'hit_obstacle': False, 'newly_discovered': 1, 'discovered_cells': 43, 'position': (3, 3)}
action=1, reward=-0.01, info={'movement_failed': False, 'hit_obstacle': False, 'newly_discovered': 0, 'discovered_cells': 43, 'position': (3, 4)}
action=1, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 43, 'position': (3, 4)}
action=3, reward=-0.26, info={'movement_failed': True, 'hit_obstacle': False, 'newly_discovered': 0, 'discovered_cells': 43, 'position': (3, 4)}
action=1, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 43, 'position': (3, 4)}
action=0, reward=-0.26, info={'movement_failed': True, 'hit_obstacle': False, 'newly_discovered': 0, 'discovered_cells': 43, 'position': (3, 4)}
action=3, reward=-0.01, info={'movement_failed': False, 'hit_obstacle': False, 'newly_discovered': 0, 'discovered_cells': 43, 'position': (3, 3)}
action=0, reward=0.03, info={'movement_failed': False, 'hit_obstacle': False, 'newly_discovered': 2, 'discovered_cells': 45, 'position': (4, 3)}
action=1, reward=0.07, info={'movement_failed': False, 'hit_obstacle': False, 'newly_discovered': 4, 'discovered_cells': 49, 'position': (4, 4)}
action=0, reward=-0.06, info={'movement_failed': False, 'hit_obstacle': True, 'newly_discovered': 0, 'discovered_cells': 49, 'position': (4, 4)}
```

Figure 3: Sample output from testing the Lunar Rover environment. Each line reports an action, reward, rover position, and transition information returned by `step()`.

Next Steps

The next step is to implement a tabular Q-learning policy and train it through repeated interactions with the Lunar Rover environment. The agent will learn action values for observed states using the rewards and transition outcomes implemented in this milestone.

A current challenge will be designing a state representation that allows the agent to distinguish newly explored terrain from previously visited terrain. The exploration reward and an epsilon-greedy action-selection policy will be used to reduce repeated movement through already explored cells and improve progress toward the goal.